Mechanics Spring 2020 Final Project on Hamiltonian Mechanics

May 4, 2020

1 Problem

1. Suppose you have a 3 kg frictionless block with a rocket to the back. The rocket applies a force that varies with the position, as defined by:

$$F_r = kx$$

where $k = 10$N/m. This is a purely 1 dimensional problem.

(a) Write out the Hamiltonian for the problem.

(b) Use Hamilton's equations to find your two equations of motion.

(c) Solve the coupled equations numerically over the course of 0.5 s for the following initial conditions:

$$A : x_o = 0 \text{ m}, p_o = 0 \text{ Ns}$$
$$B : x_o = 0 \text{ m}, p_o = 6 \text{ Ns}$$
$$C : x_o = 1 \text{ m}, p_o = 0 \text{ Ns}$$
$$D : x_o = 1 \text{ m}, p_o = 6 \text{ Ns}$$

Plot and label all trajectories in the phase space on the same plot.

(d) Confirm Louiville's theorem by comparing the area comprised between the initial points in phase space to the area between the final points in phase space. Shade both regions on your plot.

2 Background: The Hamiltonian and Hamilton's Equations

Lets start with the definition of the Hamiltonian. As was shortly touched upon in chapter 7 the definition of the Hamiltonian is $\mathcal{H} = T + U$. Much like as with the Lagrangian the Hamiltonian is a function that is central to a different formalism for classical mechanics and much like the Lagrangian is begs the question as to why it is useful and when you would use the Hamiltonian mechanics vs Lagrangian mechanics.

Lets start with the comparison to Lagrangian mechanics. Unlike the Lagrangian it is very apparent what the Hamiltonian physically represents as it is just the total energy in the system assuming "natural" coordinates. This also lead it to another big advantage as the Hamiltonian is just the total energy it is also frequently conserved which can be handy in solving problems. But as Taylor mentions in the introduction to chapter 13[1] many of the advantages are beyond the scope of this book but there is one big one that is highlighted in the book. Which is the single most important advantage of a Hamiltonian formulation over Lagrangian is the existence of Louiville's theorem for a Hamiltonian system as there is no corresponding theorem for Lagrangian mechanics[2] and this theorem will be expanded on later.

As with the Lagrangian that Hamiltonian by itself is useless without a way to translate the function into equations of motion and that is where Hamilton's equations come in. Unlike with the Lagrangian that uses the Euler-Lagrange equation to produce one second order equation of motion per degree of freedom the Hamiltonian produces 2 single order differential equations per degree of freedom. Those equations take the form of

$$\dot{q} = \frac{\partial \mathcal{H}}{\partial p} \text{ and } \dot{p} = -\frac{\partial \mathcal{H}}{\partial q}$$

[1] See Taylor pg.522 line 6

[2] See last sentence of pg.549 of Taylor

As for where these two rather odd equations come from, well the come from the more interesting definition of the Hamiltonian that I didn't mention above. This alternate Hamiltonian definition is as follows

$$\mathcal{H} = \sum_{i=1}^{n} (p_i \dot{q}_i) - \mathcal{L}$$

From this if you take the derivative of the Hamiltonian with respect to its two coordinates q, p and end up with:

$$\frac{\partial \mathcal{H}}{\partial q} = -\frac{\partial \mathcal{L}}{\partial q} = -\dot{p} \text{ and } \frac{\partial \mathcal{H}}{\partial p} = \dot{q} + p\frac{\partial \dot{q}}{\partial p} - [\frac{\partial \mathcal{L}}{\partial \dot{q}}\frac{\partial \dot{q}}{\partial p}] = \dot{q}$$

And voila you have Hamilton's equations.

3 Problem Part (a)

(a) Write out the Hamiltonian for the problem.

By the definition of the Hamiltonian we know that we need to find what the kinetic energy and the potential energy is for this system. As this is a one dimensional problem and by using a Cartesian coordinate system we know that that $T = \frac{1}{2}m\dot{x}^2$. But as Hamilton's equations are depended on the momentum it is better to express T as $T = \frac{p^2}{2m}$. Now we need to figure out what the potential of this system is. As the force given is a conservative force[3] we know it must have a potential given by $F = -\nabla U$.From that we can compute that $U = -\frac{1}{2}kx^2$. Now as we know both of those pieces we can state the Hamiltonian as:

$$\mathcal{H} = \frac{p^2}{2m} - \frac{kx^2}{2}$$

4 Problem Part (b)

(b) Use Hamilton's equations to find your two equations of motion

Starting with Hamilton's equations

$$\dot{p} = -\frac{\partial \mathcal{H}}{\partial x} \text{ and } \dot{x} = \frac{\partial \mathcal{H}}{\partial p}$$

and substituting in our Hamiltonian we have

$$\dot{p} = -\frac{\partial}{\partial x}(\frac{p^2}{2m} - \frac{kx^2}{2}) \text{ and } \dot{x} = \frac{\partial}{\partial p}(\frac{p^2}{2m} - \frac{kx^2}{2})$$

Solving for $\dot{p}$ and $\dot{x}$ we get

$$\dot{p} = kx \text{ and } \dot{x} = \frac{p}{m}$$

5 Background: My code

I will provide descriptions of how my code operates and what I did to get the results I did but and my actual code is included at the end as to not disrupt the flow of the project and to have all the code in one place.

[3] This is true as $\boldsymbol{\nabla} \times F_r = \boldsymbol{\nabla} \times kx = 0$

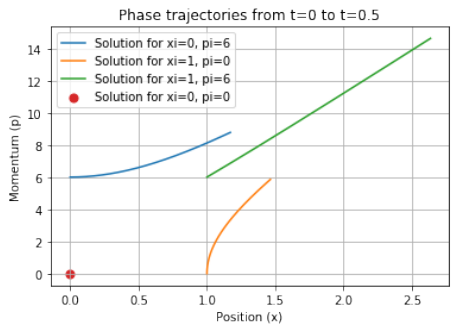
6 Problem part (c)

(c) Solve the coupled equations numerically over the course of 0.5 s for the following initial conditions:

$$A : x_o = 0 \text{ m}, p_o = 0 \text{ Ns}$$
$$B : x_o = 0 \text{ m}, p_o = 6 \text{ Ns}$$
$$C : x_o = 1 \text{ m}, p_o = 0 \text{ Ns}$$
$$D : x_o = 1 \text{ m}, p_o = 6 \text{ Ns}$$

Plot and label all trajectories in the phase space on the same plot.

To solve this problem I set up an solver using Odeint solver and then plotted them on a position-momentum phase diagram as the differential equations are based on position and momentum. Running my code it produced the following paths[4]:



Which produces the following end points

(x_i, p_i)	(x_f, p_f)
$(0, 0)$	$(0, 0)$
$(0, 6)$	$(1.1447921, 8.67850625)$
$(1, 0)$	$(1.44641771, 5.72396055)$
$(1, 6)$	$(2.59120979, 14.40246663)$

7 Background Louiville's' theorem

As mentioned at the start of this the existence of Louiville's theorem for any Hamiltonian system is an important facet of Hamiltonian mechanics. As conveniently any system that satisfies Hamilton's equations follows Louiville's theorem which I feel is part of its significance. But what is Louiville's theorem? Before getting to that lets start from the idea of a phase space. If you imagine that all the points in a particular area of this space are elements of "fluid". Louiville's theorem states
[T]he density of this "fluid" in phase space is a constant value.[5] Or equivalently that the total volume occupied by this "fluid" is a constant. While initially the value of this concept may start out obscure It is extremely important when it comes to statistical mechanics and therefore is a really important part of thermodynamics as a field. It also adds extra layers and beauty of other parts of mechanics; for example if you take a system where the location in the phase space is bounded you can get cases where the fluid boundaries would begin to wrap around itself creating interesting folding spirals.

As for some proof of this lets consider the velocity of the "fluid" in two dimensions (x, p_x). So the velocity of any point v is by definition $v = (\dot{x}, \dot{p_x})$ and from Hamilton's equations we can restate that as $v = (\frac{\partial \mathcal{H}}{\partial p}, -\frac{\partial \mathcal{H}}{\partial q})$. Taking the divergence of the velocity we get that $\boldsymbol{\nabla} \cdot v = \boldsymbol{\nabla} \cdot (\frac{\partial \mathcal{H}}{\partial p}, -\frac{\partial \mathcal{H}}{\partial q}) = 0$[6]. Therefore as the divergence of the fluid is zero then it must stay in the same volume.

[4]Note that the solution for $(x_i, p_i) = (0, 0)$ never leaves $(0, 0)$ I just added a dot so it would be visible on the graph
[5]https://hepweb.ucsd.edu/ph110b/110b$_n$otes/node93.html
[6]For a more in depth proof see pg. 549/549 in Taylor

8 Background Shoelace Algorithm

Now this background section is a bit different than all the other sections. While all the other tied directly into that mechanics being used in the problem this one is more related to the act of computation. In the next problem I am going to be confirming Louiville's theorem and as that has to do with area I need a method to compute an area given a set of points; that is where the Shoelace Algorithm comes in. What this Algorithm states is the area of a simple polygon[7] is determined by the formula:

$$\text{Area}= \frac{1}{2}|\sum_{i=1}^{n-1}(x_i y_{i+1}) + x_n y_1 - \sum_{i=1}^{n-1}(y_i x_{i+1}) - y_n x_1|$$

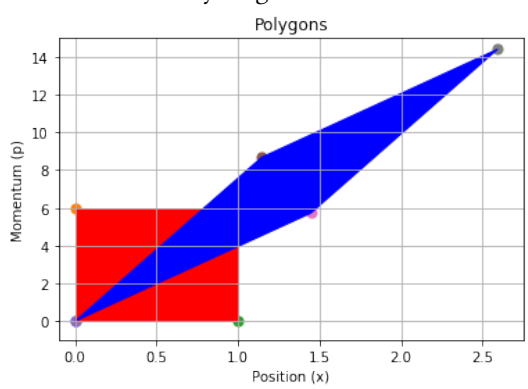
. With the i being indices defined by a consistent ordering clockwise or counter clockwise from some initial point. The reason this ordering is you are "defining the edges" using each of the points as the endpoint of the edges. While the exact reasoning behind this formula is a bit more complicated than I am presenting here it is outside the scope of the project and will be omitted for brevity.

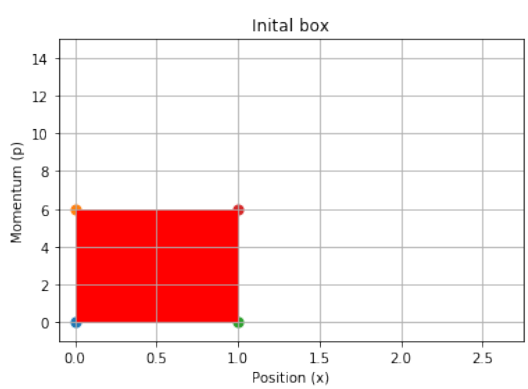
9 Problem part (d)

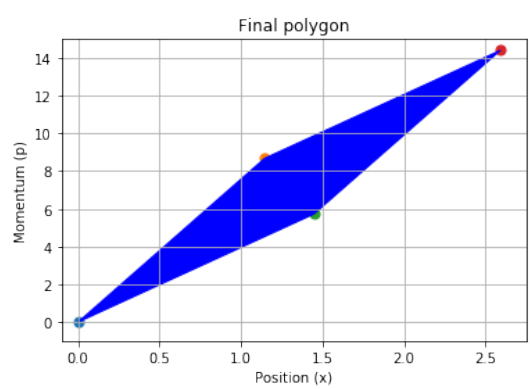
(d) Confirm Louiville's theorem by comparing the area comprised between the initial points in phase space to the area between the final points in phase space. Shade both regions on your plot.

To produce the polygons I used matplotlib.patches.Polygon as that allowed me to input the corner points as a list of ordered pairs and then it would produce the desired polygon. As for the individual polygons it is the same code but with the definition of only one of the polygons. I produced three different images, one with both of the relevant polygons overlapping as well as one for each polygon, to provide a a more complete image of the polygons.

Here are my diagrams







[7] https://mathworld.wolfram.com/SimplePolygon.html

As for my solving for the area I implemented an area solver using the above mentioned Shoelace Algorithm which produced the following results:
Initial polygon: $6.00000000000000000000\, p \times m$
Final polygon: $6.0000006281039744446844\, p \times m$
Resulting in a difference of $2.6281039744446844 \times 10^{-7}\, p \times m$

So looking at the area difference between the two polygons there is a slight area difference that shouldn't be there but as the final position of the $(x_i, p_i) = (0, 6)$ path is only accurate out to 10^{-7} so the error is right at the edge of my computational accuracy and therefore I feel it can be discounted to solver error.

10 Code reference

Relevant imports:

```
import numpy as np
import math
from scipy.integrate import odeint
from bokeh.io import output_notebook
from bokeh.resources import INLINE
from bokeh.plotting import figure, show
output_notebook(INLINE)
import matplotlib.pyplot as plt
import sympy as sym
import sympy.vector as sv
from matplotlib.patches import Polygon
#x, y, z, t = sym.symbols('x, y, z, t')
#C = sv.CoordSys3D('R')
#R=sym.symbols('R',positive=True)
```

Solver

```
def DE(state, time):
    #derivatives
    x,p =state
    #RHS of the equations
    derivs=[p/m,k*x]
    return derivs
```

```
#constants
m=3 #kg
k=10 #N/m
```

```
dt=0.001
ts = np.arange(0,0.51,dt)
Asolns=odeint(DE,[0,0],ts)
Bsolns=odeint(DE,[0,6],ts)
Csolns=odeint(DE,[1,0],ts)
Dsolns=odeint(DE,[1,6],ts)
print(Asolns[500])
print(Bsolns[500])
print(Csolns[500])
print(Dsolns[500])
```

```
[0. 0.]
[1.1447921  8.67850625]
[1.44641771 5.72396055]
[ 2.59120979 14.40246663]
```

Graphing paths

```
#solutions
plt.scatter(0,0,s=50,c='#d62728',label="Solution for xi=0, pi=0")
plt.plot(Bsolns[:,0],Bsolns[:,1],label="Solution for xi=0, pi=6")
plt.plot(Csolns[:,0],Csolns[:,1],label="Solution for xi=1, pi=0")
plt.plot(Dsolns[:,0],Dsolns[:,1],label="Solution for xi=1, pi=6")

#graphical settings
plt.xlabel('Position (x)')
plt.ylabel('Momentum (p)')
plt.title('Phase orbits for until t=0.5')
plt.grid(True)
plt.legend()
plt.show
```

Polygon definition & graphing

```
fig, ax = plt.subplots()

#polygon definition
pointsinital=[[0,0],[0,6],[1,6],[1,0]]
pointsfinal=[[Asolns[500,0],Asolns[500,1]],[Bsolns[500,0],Bsolns[500,1]],
             [Dsolns[500,0],Dsolns[500,1]],[Csolns[500,0],Csolns[500,1]]]

Initialpoly=Polygon(pointsinital,color='red')
ax.add_patch(Initialpoly)

Finalpoly=Polygon(pointsfinal,color='blue')
ax.add_patch(Finalpoly)

#inital point definition
plt.scatter(0,0,s=50,label="x=0, pi=0")
plt.scatter(0,6,s=50,label="x=0, pi=6")
plt.scatter(1,0,s=50,label="x=1, p=0")
plt.scatter(1,6,s=50,label="x=1, p=6")

#final point definition
plt.scatter(Asolns[500,0],Asolns[500,1],s=50,label="xi=0, pi=0")
plt.scatter(Bsolns[500,0],Bsolns[500,1],s=50,label="xi=0, pi=0")
plt.scatter(Csolns[500,0],Csolns[500,1],s=50,label="xi=0, pi=0")
plt.scatter(Dsolns[500,0],Dsolns[500,1],s=50,label="xi=0, pi=0")

#graphical settings
plt.xlabel('Position (x)')
plt.ylabel('Momentum (p)')
plt.title('Polygons')
plt.grid(True)
plt.xlim(-.1, 2.75)
plt.ylim(-1, 15)
plt.show()
```

Area computation:

```python
#Area solver function def
def Area(x1,y1,x2,y2,x3,y3,x4,y4):

    result=0.5*abs(((x1*y2+x2*y3+x3*y4+x4*y1)-(x2*y1+x3*y2+x4*y3+x1*y4)))

    return result
```

```python
A_0=Area(0,0,0,6,1,6,1,0)
A_1=Area(0,0,1.1447921,8.67850625,2.59120979, 14.40246663,1.44641771, 5.72396055)
Ad=A_1-A_0
print('Area of inital polygon')
print(A_0,)
print('Area of final polygon')
print(A_1,)
print('Difference')
print(Ad,)
```

```
Area of inital polygon
6.0
Area of final polygon
6.0000002628103974
Difference
2.6281039744446844e-07
```